

第 33 章: Exception Basics (异常基础)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是全书"异常处理"部分的开篇, 也是主教程的最后一章。前 32 章你已经学会了数据、语句、函数、模块、类这些"静态"构建块, 现在要处理程序运行中的"意外"——异常(exception)。作者故意把这一主题推迟到现在, 因为"自己定义异常"需要用到类(class)的知识。本章只讲异常的基础: 为什么要用异常、异常起什么作用, 以及 try/except/finally/raise/assert 五大工具的初步用法, 细节留到第 34 章。

开篇引言

英文原文

This part of the book deals with exceptions—events that signal conditions and modify the flow of control through a program. In Python, exceptions are triggered automatically on errors, and they can be both triggered and intercepted by your code. They are processed by four statements we'll study here, the first of which comes in multiple flavors that qualify as different statement forms by some measures: - try / except / el

se / finally — Catch and recover from exceptions raised by Python, or by you- raise — Trigger an exception manually in your code- assert — Conditionally trigger an exception in your code- with — Use context managers that automate exception handling We've met some of these briefly before, but full coverage of this topic was saved until the end of the main part of this book because you need to know about classes to code exceptions of your own. Still, with a few exceptions (pun intended), you'll find that exception handling is simple in Python because it's integrated into the language itself as another high-level tool. Before we dig into the "how," though, let's get clear on the "why."

中文翻译

本书的这一部分讨论异常 (exception) ——用于发出状态信号、修改程序控制流的"事件"。在 Python 中, 异常会在出错时自动触发, 而且你的代码既可以主动触发异常, 也可以拦截异常。异常由我们接下来要学习的四条语句处理, 其中第一条还带有多种变体, 按某些标准算得上不同的语句形式: - try / except / else / finally——捕获并恢复由 Python 或由你引发的异常- raise——在代码中手动触发异常- assert——在代码中有条件地触发异常- with——使用自动处理异常的上下文管理器 (context manager) 我们之前已经简单接触过其中一些, 但这一主题的完整讲解被留到本书主体部分的结尾, 因为编写自己的异常需要先了解类 (class)。尽管如此, 除了一些特例 (双关语: with a few exceptions 字面意思就是"有几个异常"), 你会发现 Python

的异常处理非常简单，因为它作为又一个高级工具被集成进了语言本身。不过，在深入"怎么做"之前，让我们先搞清楚"为什么"。

深度理解

·核心概念：异常是一种控制流（control flow）装置——它本身不是错误，而是一种"事件"：程序运行中信号（signal）条件发生变化、并借以跳转控制流的机制。Python 在运行期检测到错误时自动"引发"（raise）异常；你的代码既可以被动捕获它，也可以主动引发它。

·底层实现：异常在 CPython 底层对应 PyErrSetObject/PyErrSetString 一类的 C API 调用与线程级的"错误指示器"（error indicator），解释器在字节码层面用专门的指令记录"活动异常处理器"的位置。当异常被抛出后，CEval 主循环逐层"弹出"（unwind）调用栈，寻找匹配的处理器，找不到就交给顶层默认处理器。这些细节第 34 章会展开。

·设计原因：作者把异常推迟到全书末尾讲，是因为"自定义异常需要类"。而异常本身被设计为语言内置的一等工具，而不是像 C 那样靠程序员手动检查返回值——这能让代码专注于"正常路径"，错误处理只出现在需要恢复的地方。

·实际问题：服务器程序不能因为一次内部错误就退出；文件、数据库连接必须在出错时也能关闭。没有异常机制，这些需求只能靠每个函数都检查返回值来实现，代码会淹没在错误检测里。

·初学者误区：以为 try 是"语句"，其实按作者的衡量标准 try 的四种变体 (try/except、try/finally、try/except/else、try/except/finally/else) 算作不同的语句形式。另外别混淆"异常 (exception)"与"错误 (error)"——异常是语言工具，错误只是触发它的常见原因之一；合法条件 (如搜索失败) 也可以引发异常。

为什么要使用异常？ (Why Use Exceptions?)

英文原文

In a nutshell, exceptions let us jump out of arbitrarily large chunks of a program. Consider the hypothetical pizza-making robot we discussed earlier in the book. Suppose we took the idea seriously and actually built such a machine. To make a pizza, our culinary automaton would need to execute a plan, which we would implement as a Python program: it would take an order, prepare the dough, add toppings, bake the pie, and so on. Now, suppose that something goes very wrong during the "bake the pie" step. Perhaps the oven is broken, or perhaps our robot miscalculates its reach and spontaneously combusts. Clearly, we want to be able to jump to code that handles such unusual states quickly. As we have no hope of finishing the pizza task in such unusual cases, we might as well abandon the entire plan. That's exactly what exceptions let you

r programs do: they can jump to an exception handler in a single step, abandoning all activity begun since the exception handler was entered. Code in the exception handler can then respond to the raised exception as appropriate (by calling the fire department, for instance!). One way to think of an exception is as a sort of structured "go-to." An exception handler (try statement) leaves a marker and executes some code. Somewhere further ahead in the program, an exception is raised that makes Python jump back to that marker, abandoning any code that was started and functions that were called after the marker was left. The net effect unwinds the program's control flow back to the marker and resumes there. This protocol provides a coherent way to respond to unusual events. Moreover, because Python jumps to the handler statement immediately, your code is simpler—there is usually no need to check status codes after every operation and function call that could possibly fail. Instead, we catch errors only where we need to recover from them.

中文翻译

简而言之，异常让我们能够跳出程序中任意大小的代码块。回想一下本书前面讨论过的“做披萨的机器人”这个假想例子。假设我们认真对待这个想法，真的造出了这样一台机器。要做披萨，我们的“烹饪自动机”需要执行一个计划，这个计划我们会用 Python 程序来实现：它要接收订单、准备面团、添加配料、烘烤披萨，等等。现在假设“烘烤披萨”这一步

出了大问题。也许烤箱坏了，也许我们的机器人算错了自己的臂展而自发燃烧（spontaneously combusts）。显然，我们希望能快速跳到处理这种异常状态的代码。既然在这种情况下完成披萨任务已无希望，我们还不如彻底放弃整个计划。这正是异常让程序做到的事情：程序可以一步跳到异常处理器（exception handler），放弃自进入该处理器以来启动的所有活动。异常处理器中的代码随后可以针对引发的异常做出适当的响应（比如拨打消防队电话！）。把异常看作一种"结构化的 goto"是个不错的理解方式。异常处理器（try 语句）留下一个标记（marker）并执行一些代码。程序更靠后的某个位置引发了异常，使 Python 跳回那个标记，放弃标记之后启动的任何代码和调用的任何函数。净效果就是：把程序的控制流"回卷"（unwind）到标记处，并从这里继续执行。这个协议提供了一种连贯一致的方式来响应不寻常的事件。此外，由于 Python 会立即跳到处理器语句，你的代码也更简单——通常不需要在每个可能失败的操作和函数调用之后检查状态码。相反，我们只在需要恢复的地方捕获错误。

深度理解

·核心概念：异常的本质是结构化跳转——"任意大代码块"意味着异常可以跨过任意多层函数调用直接跳到处理器，这是 break/continue 做不到的（它们只能跳出循环这一层）。

·底层实现：所谓"回卷"（unwind）在底层就是栈帧弹出：异常发生时，CPython 沿调用栈向上逐帧搜索 handler，途中被弹出的函数栈帧（frame）里的局部变量随之销毁。

毁。这也是为什么异常跳转后"回不去"——栈已经拆了。

·设计原因：为什么不用返回值检查错误？因为"每个调用点都要检查"把错误处理的责任分散到全程序各处，而异常把责任集中到"需要恢复的地方"。作者用"披萨机器人自燃"这个夸张例子说明：意外事件根本不在主流程里，不该用主流程的逻辑去处理。

·实际问题：搜索例程失败、网络请求超时、文件不存在...这些"非主路径"的情况若用 if 层层检查，主流程会面目全非；异常让主流程保持"假设一切正常"的书写方式。

·初学者误区：以为"异常 = 崩溃"。实际上异常只是跳转工具，跳到处理器后程序可以正常继续；只有不捕获的异常才最终走向崩溃。另一个误区是以为异常处理有巨大性能开销——异常路径慢，但正常路径零开销（这一点本章末尾的"错误检查"专栏有印证）。

异常的角色 (Exception Roles)

英文原文

In less hypothetical programs, exceptions serve a variety of purposes. Here are some of their most common roles: Error handling — Python raises exceptions whenever it detects errors in programs at runtime. You can catch and respond to the errors in your code, or ignore the exceptions that are raised. If an error is ignored, Python's default exception-handling behavior kicks in: it stops the program and prints an error mess

age. If you don't want this default behavior, code a `try` statement to catch and recover from the exception —Python will jump to your `try` handler when the error is detected in the statement's code, and your program will resume execution after the `try`. Event notification — Exceptions can also be used to signal valid conditions without you having to pass result flags around a program or test them explicitly. For instance, a search routine might raise an exception on failure, rather than returning an integer result code—and hoping that the code will never be a valid result. Special-case handling — Sometimes a condition may occur so rarely that it's hard to justify convoluting your code to handle it in multiple places. You can often eliminate special-case code by handling unusual cases in exception handlers in higher levels of your program. An `assert` can similarly be used to check that conditions are as expected during development. Termination actions — As you'll see, the `finally` option in a `try` statement allows you to guarantee that required closing-time operations will be performed, regardless of the presence or absence of exceptions in your programs. The `with` statement offers an alternative in this department for objects that support its expected method-call protocol. Unusual control flows — Finally, because exceptions are a sort of high-level and structured "go-to," you can use them as the basis for implementing exotic control flows. For instance, although the language do

es not explicitly support backtracking, you can implement it in Python by using exceptions and logic to unwind assignments.¹ There is no "go to" statement in Python (thankfully) and no built-in backtracking (today), but exceptions can sometimes serve similar roles; a raise, for instance, can be used to jump out of multiple loops in ways that break cannot. We saw some of these roles briefly earlier and will study typical exception use cases in action later in this part of the book. For now, let's get started with a look at Python's exception-processing tools.

中文翻译

在不太虚构的程序中，异常有各种各样的用途。以下是它们最常见的几种角色：错误处理（Error handling）——只要 Python 在运行期检测到程序中的错误，它就会引发异常。你可以在代码中捕获并响应这些错误，也可以忽略被引发的异常。如果错误被忽略，Python 的默认异常处理行为就会启动：停止程序并打印错误消息。如果你不想要这种默认行为，就编写 try 语句来捕获并从异常中恢复——当 try 语句的代码中检测到错误时，Python 会跳到你的 try 处理器，程序恢复执行 try 之后的代码。事件通知（Event notification）——异常也可以用来发出“条件合法”的信号，而不必在程序中到处传递结果标志（result flag）或显式地测试它们。例如，一个搜索例程可以在失败时引发异常，而不是返回一个整数结果码——还得祈祷这个结果码永远不会恰好是合法结果。特殊情况处理（Special-case handling）——有时某种情况极其罕见，很难有理由为了让多个地方处理

它而把代码搞得错综复杂。你可以把不寻常的情况交给程序更高层的异常处理器，从而消除特殊情况的代码。assert 语句同样可以用于在开发期间检查条件是否符合预期。终止动作 (Termination actions) ——正如你将看到的，try 语句中的 finally 选项允许你保证必需的"收尾"操作一定被执行，无论程序中是否有异常。with 语句为支持其预期方法调用协议的对象提供了这方面的替代方案。非常规控制流 (Unusual control flows) ——最后，因为异常是一种高层级的、结构化的 "goto"，你可以把它们作为实现奇特的控制流的基础。例如，尽管语言并不显式支持回溯 (backtracking)，但你可以在 Python 中通过异常和逻辑回卷赋值来实现它。¹ Python 没有 goto 语句 (谢天谢地)，也没有内置的回溯 (目前如此)，但异常有时可以扮演类似的角色；例如，一个 raise 可以以 break 无法做到的方式跳出多层循环。我们之前已经简单见过其中一些角色，在本部分的后面将研究实际使用中典型的异常用例。现在，让我们开始看看 Python 的异常处理工具。

深度理解

·核心概念：异常有五大用途——错误处理、事件通知、特殊情况处理、终止动作、非常规控制流。前两种最容易混淆：错误处理是"对失败的反应"，事件通知是"用异常替代返回值标志"。

·底层实现："结果码撞车"是真实工程问题——C 语言里 -1 既是错误码也可能是合法结果 (如 index() 返回 -1)；异常对象 (exception instance) 是独立的数据载体，永远不可能与合法业务结果混淆。这就是为什么作者说"祈祷

那个码永远不是合法结果"——返回码方案本质上是不可靠的。

·设计原因：Python 刻意不提供 goto 和内置回溯（Prolog/Icon 有），因为"结构化 goto"（异常）在覆盖 99% 跳转需求的同时，不会像裸 goto 那样把程序变成意大利面。用异常模拟回溯（如第 20 章的生成器结合异常）是高级玩法，属于"允许但不鼓励"的范畴。

·实际问题：写健壮代码的三件套——外部依赖要 try/except（错误处理）、稀缺资源要 finally/with（终止动作）、开发期断言要 assert（特殊情况检查）。这三者覆盖了真实程序几乎全部异常场景。

·初学者误区：以为 assert 是给用户看错误信息的——错了，assert 是开发期的自检工具，python -O 优化模式下会被整体删除（后面章节会讲）；生产环境面向用户的输入校验应该用显式 if 或 raise。另外注意本章答案里说的：异常还能"减少错误检查代码量"——因为所有错误自动"上浮"到处理器。

异常简述 (Exceptions: The Short Story)

英文原文

Compared to some other core language topics we've explored in this book, exceptions are a fairly lightweight tool in Python. Because they are so simple, let's jump right into some code.

中文翻译

与我们在本书中探索的其他核心语言主题相比，异常在 Python 中是一个相当轻量级的工具。因为它们如此简单，让我们直接进入代码。

深度理解

·核心概念：异常工具虽然承载"错误处理"这么严肃的职责，但使用上只有几个关键字——这正是"简单工具解决复杂问题"的 Python 哲学。

·底层实现："轻量"体现在字节码上：try 块只需在编译期记录处理器跳转地址，正常执行路径只是一条条普通指令，没有任何额外判断——这就是"零开销"的由来。

·设计原因：作者用整章篇幅反复强调"简单"，是想化解读者（尤其是有 C/Java 经验的读者）对异常的心理负担——Python 把最常用的机制压到最少的语法里。

·实际问题：轻量意味着可以"到处用"：小到一个索引访问，大到整个程序体，都可以包进 try。

·初学者误区：把异常处理复杂化——例如处处 try、层层嵌套。本章后面会展示：多数情况下你只需在"需要恢复的地方"包一层。

默认异常处理器 (Default Exception Handler)

英文原文

Suppose we write the following function in the interactive REPL of our choice:

```
python >> def fetcher(obj, index):
```

```
... return obj[index]
```

There's not much to this function—it simply indexes an object on a passed-in index. In normal operation, it returns the result of a legal index:

```
python >> food = 'pizza' >> fetcher(food, 4) # Like x[4], last item 'a'
```

However, if we ask this function to index off the end of the string, an exception will be triggered when the function tries to run `obj[index]`. Python detects out-of-bounds indexing for sequences and reports it by raising (triggering) the built-in `IndexError` exception:

```
python >> fetcher(food, 5) # Default handler – console interface
Traceback (most recent call last): File "<stdin>", line 1, in <module>
File "<stdin>", line 2, in fetcher
IndexError: string index out of range
```

Because our code does not explicitly catch this exception, it filters back up to the top level of the program and invokes the default exception handler, which simply prints the standard error message shown here.

By this point in the book, you've probably seen your share of standard error messages. They include the exception that was raised, along with a stack trace—a list of all the lines and functions that were active whe

n the exception occurred, which has been largely omitted in this book for space and brevity.

The error message text here was printed by Python 3.12 in a console. It can vary slightly per release, and even per interactive REPL, so you shouldn't rely upon its exact form—in either this book or your code. When you're coding interactively in a console interface, the filename may be just "<stdin>," meaning the standard input stream.

When working in the IDLE GUI's interactive shell today, though, the filename is "<pyshell...>," and source lines are displayed, too. Either way, file line numbers are not very meaningful when there is no file (you'll see more interesting error messages later in this part of the book):

```
python >> fetcher(food, 5) # Default handler – IDLE GUI interface
Traceback (most recent call last): File "<pyshell#3>", line 1, in <module>
fetcher(food, 5) File "<pyshell#0>", line 2, in fetcher
```

```
return obj[index]
IndexError: string index out of range
```

In a more realistic program launched outside the interactive REPL, after printing an error message the default handler at the top also terminates the program immediately. That course of action makes sense for simple scripts; errors often should be fatal, and the best you can do when they occur is inspect the standard

error message.

中文翻译

假设我们在自己选定的交互式 REPL 中编写下面的函数：python >>> def fetcher(obj, index): ... return obj[index] 这个函数没什么内容——它只是按传入的下标 (index) 对对象进行索引。正常操作时，它返回合法索引的结果：python >>> food = 'pizza'>>> fetcher(food, 4) # 相当于 x[4]，最后一个字符'a'然而，如果我们让这个函数对字符串"越界"索引，当函数试图执行 obj[index] 时就会触发异常。Python 会检测序列的越界索引，并通过引发 (raising, 即触发) 内置的 IndexError 异常来报告它：python >>> fetcher(food, 5) # 默认处理器——控制台界面Traceback (most recent call last): File "<stdin>", line 1, in <module> File "<stdin>", line 2, in fetcher IndexError: string index out of range 因为我们的代码没有显式捕获这个异常，它一路"上浮" (filters back up) 到程序的顶层，并调用默认异常处理器 (default exception handler)，后者只打印这里所示的标准错误消息。读到这里，你大概已经见过不少标准错误消息了。它们包含被引发的异常，以及一条堆栈回溯 (stack trace) ——异常发生时所有处于活动状态的代码行和函数的列表，本书出于篇幅和简洁的考虑大多省略了它。这里的错误消息文本是由 Python 3.12 在控制台中打印的。不同版本之间甚至不同交互式 REPL 之间，它的形式都可能略有差异，所以无论在本书里还是在你自己的代码中，都不应该依赖它的精确格式。当你在控制台界面中交互式编码时，文件名可能只是 "<stdin>"，意思是标准输入流。不过，如今在 IDLE 图形界面的交互式 shell

中工作时，文件名是 "<pyshell...>"，而且还会显示源代码行。无论哪种方式，在没有文件的情况下，文件行号意义都不大（本部分后面你会看到更有意思的错误消息）：`python >>> fetcher(food, 5) # 默认处理器——IDLE 图形界面`
`Traceback (most recent call last): File "<pyshell#3>", line 1, in <module> fetcher(food, 5)`
`File "<pyshell#0>", line 2, in fetcher return obj[index]`
`IndexError: string index out of range` 在交互式 REPL 之外启动的更真实的程序中，打印完错误消息后，顶层的默认处理器还会立即终止程序。这对简单脚本来说是合理的做法；错误往往应该是致命的，它们发生时你所能做的最好的事情就是检查标准错误消息。

深度理解

·核心概念：未被捕获的异常会逐层向上传播（propagation），最终到达顶层并触发默认异常处理器——打印 traceback（堆栈回溯）并终止程序。这是你“必须亲眼见过”的 Python 行为。

·底层实现：堆栈回溯其实就是调用栈的真实快照——`File "<stdin>", line 2, in fetcher` 一行对应一个栈帧（frame）。CPython 在异常对象上保存 traceback，打印时沿链表把每个帧的行号、文件名、函数名、源代码行依次列出。在控制台里没有文件，所以文件名是伪装的 "<stdin>"；IDLE 则显示 "<pyshell#N>"。

·设计原因：为什么简单脚本要让错误“致命”？因为脚本没有长期运行的义务——出了错最合理的动作就是打印清晰的诊断信息然后退出，把排查责任交给开发者。服务器程

序才需要"带病运行"（用 try 恢复）。

·实际问题：调试的第一技能就是读懂 traceback：从最上面一行的调用者往下读到最下面一行（异常发生点）。IndexError: string index out of range 末尾的"异常类型：消息"是最后一行，也是最关键的一行。

·初学者误区：以为错误消息的文本格式是 API——作者明确警告"不要依赖它的精确形式"（Python 3.12 与早期版本、控制台与 IDLE 输出都不同）。还有人不看 traceback 就盲改代码——注意 traceback 的阅读顺序是"从下往上看"更高效：先看异常类型与消息，再往回追调用链。

代码分析

```
def fetcher(obj, index):  
    return obj[index]          #  
  
food = 'pizza'  
fetcher(food, 4)              # x[4] 'a'  
fetcher(food, 5)              # IndexError
```

解释器如何处理：执行 fetcher(food, 5) 时，Python 先创建一个新的函数栈帧（frame），把 obj 绑定到字符串'pizza'、index 绑定到 5。执行 obj[index] 时，字节码 BINARYSUBSCR 调用字符串的 getitem 方法，内部检查发现索引 5 超出 0~4 的范围，于是 CPython 调用 PyErr_SetString(PyExcIndexError, "string index out of range") 设置错误指示器。此时没有匹配的处理器，解释器开始栈展开（stack unwinding）：fetcher 的帧被销毁，回到模块层；模块层也没有处理器，异常继续上浮到顶层，触发默认处理器——打印 traceback（异常类型、文件名、

行号、函数名逐帧列出) 并终止进程。

设计思想：索引越界被设计成"异常"而不是"返回 None"，体现了 Python 的"失败要响亮" (fail loudly) 哲学——静默返回错误值会掩盖 bug。而 traceback 从"调用者到被调用者"的打印顺序，恰好让你能顺着读完整条调用链。

捕获异常 (Catching Exceptions)

英文原文

Sometimes, though, program termination on exceptions isn't what you want. Server programs, for instance, typically need to remain active even after internal errors. If you don't want the default exception behavior, wrap the call in a try statement to catch exceptions yourself (copy/pasters: omit the "..." here per the note ahead):

```
python >>> try: ... fetcher(food, 5) ... except IndexError: # Catch and recover ... print('got exception') ... got exception
```

>>> Now, Python automatically jumps to your handler—the block under the except clause that names the exception raised—when an exception is triggered while the try block is running. The net effect is to wrap a nested block of code in an error handler that intercepts the block's exceptions. When working interactively

y like this, after the except clause runs, we wind up back at the Python prompt. In a more realistic program, try statements not only catch exceptions but also recover from them:

```
python >>> def catcher(): ... try: ... fetcher(food, 5) ...  
except IndexError: ... print('got exception') # Catch and  
recover more ... print('continuing') >>> catcher() got  
exception continuing
```

>>> This time, after the exception is caught and handled, the program resumes execution after the entire try statement that caught it—which is why we get the "continuing" message here. We don't see the standard error message, and the program continues on its way normally. Notice, though, that there's no way in Python to go back to the code that triggered the exception (short of rerunning the code that reached that point all over again, of course). Once you've caught the exception, control continues after the entire try that caught the exception, not after the statement that kicked off the exception. In fact, Python clears the memory of any functions that were exited as a result of the exception, like `fetcher` in our example; their variables are discarded, and they're not resumable. The try both catches exceptions and is where the program resumes. Python does not, however, undo any work done by the try block up to the point where the exception occurred—any changes made to referenced mutable objects and accessible global names live on. This is

sn't a problem if it's known:

```
python >>> L, S = [], 'text' >>> def modder(): ... L.append('added') # Change a mutable ... global S; S = 'changed' # Change a global ... fetcher(food, 5) # Trigger an exception >>> try: ... modder() ... except IndexError: ... print('got exception') ... got exception

>>> L, S # Changes retained (['added'], 'changed')
```

As you'll see later in this part of the book, the try can also use except clauses to process multiple exceptions, but this is a convoluted extension with narrow scope that doesn't play well with others, and you can safely defer studying until you've mastered the fundamentals. > NOTE (Presentation note) : The interactive REPL's "..." continuation prompt reappears in this part for some top-level try statements, because their code won't work if copied and pasted unless nested in a function or class (the except and other lines must align with the try, and not have extra preceding spaces that are needed to illustrate their indentation structure here). To run, simply type or paste statements with "..." prompts one line at a time, and without their leading "..." prompts.

中文翻译

不过，有时“遇到异常就终止程序”并不是你想要的。例如，服务器程序通常需要在发生内部错误后依然保持运行。如果你不想要默认的异常行为，就把调用包进一个 try 语句，自

已捕获异常（复制粘贴党注意：根据后面的说明省略这里的 "...")：python >>> try: ... fetcher(food, 5) ... except IndexError: # 捕获并恢复... print('got exception') ... got exception >>> 现在，当 try 块运行期间触发异常时，Python 会自动跳到你的处理器（handler）——即命名了所引发异常的 except 子句下面的代码块。净效果就是把一段嵌套代码包进一个错误处理器，由它拦截（intercepts）这段代码的异常。像这样交互式工作时，except 子句执行完后，我们会回到 Python 提示符。在更真实的程序中，try 语句不仅能捕获异常，还能从中恢复（recover）：python >>> def catcher(): ... try: ... fetcher(food, 5) ... except IndexError: ... print('got exception') # 更进一步：捕获并恢复... print('continuing') >>> catcher() got exception continuing >>> 这一次，异常被捕获并处理后，程序从捕获它的整个 try 语句之后继续执行——这就是为什么这里会打印 "continuing"。我们看不到标准错误消息，程序正常继续它的旅程。不过请注意，Python 中没有办法回到触发异常的代码那里（当然，除非把到达那一点的代码整个重新运行一遍）。一旦你捕获了异常，控制流就继续到捕获异常的整个 try 语句之后，而不是继续到引发异常的那条语句之后。事实上，Python 会清除因异常而退出的任何函数的内存，比如我们例子中的 fetcher；它们的变量被丢弃，并且不可恢复。try 既捕获异常，也是程序恢复执行的地方。然而，Python 并不会撤销 try 块在异常发生之前所做的任何工作——对引用的可变对象（mutable object）的修改和对可访问的全局名字（global name）的改动都会保留下来。只要事先知道这一点，就不算问题：python >>> L, S = [], 'text' >>> def modder(): ... L.append('added') # 修改

可变对象... `global S; S = 'changed' # 修改全局变量... fetcher(food, 5) # 触发一个异常>>> try: ... modder() ... except IndexError: ... print('got exception') ... got exception >>> L, S # 改动被保留(['added'],'changed')` 正如你将在本部分后面看到的, `try` 还可以使用 `except` 子句来处理多个异常, 但这是一个复杂且适用范围狭窄的扩展, 与其他特性配合不佳, 你可以放心地推迟到掌握基础之后再学。

> 注意 (呈现说明): 交互式 REPL 的 `"..."` 续行提示符在本部分再次出现在一些顶层的 `try` 语句中, 因为如果直接复制粘贴, 除非嵌套在函数或类里, 否则这些代码无法运行 (`except` 等行必须与 `try` 对齐, 不能有这里为了展示缩进结构而加的前导空格)。要运行它们, 只需把带 `"..."` 提示符的语句一行一行地输入或粘贴, 并且去掉前面的 `"..."` 提示符即可。

深度理解

·核心概念: `try/except` 是错误恢复机制: 处理器执行后, 程序从"整个 `try` 之后"继续, 而不是从出错点继续。捕获异常 $\neq$ 撤销错误——中间层函数的工作已经白做, 局部变量已被丢弃。

·底层实现: `try` 块编译成字节码时会登记一个"异常处理器表项" (在 3.11+ 中为 `SETUPFINALLY / exception table` 机制)。异常发生时, `CEval` 循环查找当前帧的异常表, 找到匹配的处理器后设置指令指针并跳转; 没有匹配就销毁当前帧, 把异常转发给调用者帧。`modder()` 里 `fetcher` 的帧被丢弃, 所以 `fetcher` 的局部变量消失; 但 `L`、`S` 是外层作用域对象, 异常只销毁"栈帧内存", 不销毁"堆上

的对象"——所以列表 L 的追加和全局 S 的重绑定都保留了下来。

·设计原因：为什么不支持"回到出错点继续"？因为 Python 是半自动事务式设计：栈回卷（撤销调用）成本低，而"回滚堆上对象"成本不可控（Python 无法知道怎么撤销一次列表追加）。这正是脚注 1 说的"Python 异常不是真正的回溯（backtracking）"——回溯需要撤销计算，Python 只丢弃栈。

·实际问题：except 是 Python 3.11 新增的多异常处理语法（配合 ExceptionGroup），作者直言它"复杂、范围窄、与其它特性配合不佳"，建议初学者推迟学习——这本身就是一条宝贵的工程判断：地基没打牢之前，不要碰花哨语法。

·初学者误区：以为捕获异常后"程序就从出错处接着跑"——不会！异常点的所有后续语句都不会执行，函数调用栈已经被拆除。另一个误区是把 try 当"事务回滚"——L.append('added') 这类已生效的修改不会自动撤销。还有一个 REPL 实操坑：把书里带 ... 提示符的代码整段粘贴会报缩进错误（IndentationError），必须逐行输入或去掉提示符。

代码分析

```
def catcher():
    try:
        fetcher(food, 5)          # IndexError
    except IndexError:            #
        print('got exception')    #
    print('continuing')          # try
```

```
L, S = [], 'text'
def modder():
    L.append('added')             #
    global S; S = 'changed'      #
    fetcher(food, 5)             #

try:
    modder()
except IndexError:
    print('got exception')

L, S                               # (['added'], 'changed')
```

解释器如何处理：调用 `catcher()` 时进入新帧。执行到 `fetcher(food, 5)` 时 `BINARYSUBSCR` 抛出 `IndexError`，`CEval` 主循环检查当前帧的异常表——发现第 1~3 行有一个 `except IndexError` 处理器，于是把栈顶（被弹出的 `fetcher` 帧已销毁）弹出异常对象，跳转到 `except` 子句开始执行 `print('got exception')`。处理器执行完毕后，指令指针跳到整个 `try` 复合语句之后，执行 `print('continuing')`。注意：跳转目标永远是“整个 `try` 之后”，而不是 `try` 块内出错的那一行——这就是“没有回头路”的字节码层面的真相。`modder` 例子里，异常表在模块层，`modder` 和 `fetcher` 两个帧都被丢弃，但 `L` 指向的列表对象和全局名字 `S` 都还在堆上，因此输出 `(['added'], 'changed')`。

设计思想：`except` 类型指定“想捕获什么”，其余异常照旧

上浮——这叫做“精确捕获”。只要不写裸 `except:`，就能避免把 `KeyboardInterrupt` 之类的系统异常也吞掉。

引发异常 (Raising Exceptions)

英文原文

So far, we've been letting Python raise exceptions for us by making mistakes (on purpose this time!), but our scripts can raise exceptions too—that is, exceptions can be raised by Python or by your program, and can be caught or not. To trigger an exception manually, simply run a `raise` statement. User-triggered exceptions are caught the same way as those Python raises. The following may not be the most useful Python code ever penned, but it makes the point—raising the built-in `IndexError` exception:

```
python >> try:
```

```
... raise IndexError # Trigger exception manually ...  
except IndexError: ... print('got exception') ...  
got exception
```

As usual, if they're not caught, user-triggered exceptions are propagated up to the top-level default exception handler and terminate the program with a standard error message:

```
python >> raise IndexError  
Traceback (most recent call last):  
File "<stdin>", line 1, in <module> IndexError
```

As you'll see in the next chapter, the `assert` statement can be used to trigger exceptions, too—it's a conditional raise predicated on a test, used mostly for debugging purposes and sanity checks during development:

```
python >> assert 1 < 0, 'Not in this universe!'
Traceback (most recent call last): File "<stdin>", line 1, in <module>
AssertionError: Not in this universe!
```

Also in the next chapter, you'll learn that `raise` can use a `from` clause to "chain" exceptions; generally speaking, this is not common, but can be used to give more context where it's useful.

中文翻译

到目前为止，我们一直是靠制造错误（这次是故意的！）让 Python 替我们引发异常，但我们的脚本自己也能引发异常——也就是说，异常既可由 Python 引发，也可由你的程序引发，并且可以被捕获也可以不被捕获。要手动触发异常，只需执行一条 `raise` 语句。用户触发的异常与 Python 引发的异常以同样的方式被捕获。下面这段也许不是人类写过的最有用的 Python 代码，但它说明了一个要点——引发内置的 `IndexError` 异常：`python >>> try: ... raise IndexError # 手动触发异常... except IndexError: ... print('got exception') ... got exception` 和往常一样，如果用户触发的异常未被捕获，它会被传播到顶层的默认异常处理器，并以一条标准错误消息终止程序：`python >>> raise IndexE`

rror Traceback (most recent call last): File "<stdin>", line 1, in <module> IndexError 正如你将在下一章看到的，`assert` 语句也可以用来触发异常——它是一个以测试为前置条件的条件式 `raise`，主要用于调试和开发期间的健全性检查 (sanity check)：python >>> `assert 1 < 0, 'Not in this universe!'` Traceback (most recent call last): File "<stdin>", line 1, in <module> AssertionError: Not in this universe! 同样在下一章，你会学到 `raise` 可以使用 `from` 子句来“链接” (chain) 异常；一般来说这并不常见，但在有用武之地时可以为异常提供更多上下文。

深度理解

·核心概念：`raise` 把“引发异常”从解释器特权变为程序员特权——你的代码可以随时制造一个与真实错误完全无法区分的异常（它们走同一条传播/捕获管道）。`assert` 则是带条件的 `raise`：`assert 条件, 消息` 等价于“条件为假就 `raise AssertionError(消息)`”。

·底层实现：`raise` 编译为 `RAISEVARARGS` 字节码，它直接构造异常对象并交给异常分发机制——与 `BINARYSUBSCR` 触发 `getitem` 错误走的是同一条路径。`assert` 编译为“先执行条件判断，为假则跳转到 `raise` 分支”的几条指令；注意在 `-O` 优化模式下这些指令会被整体剥离。

·设计原因：为什么需要手动 `raise`？因为“合法但特殊”的情形（参数非法、未找到、状态不允许）需要一种统一的错误报告语言——让调用方通过 `except` 捕获，而不是靠猜测返回值的含义。

·实际问题：raise 是库设计的关键工具——你的函数应该“快速失败”（fail fast）：发现非法输入立即 raise 带清晰消息的 ValueError，而不是返回 None 让 bug 潜伏。assert 则用于开发期不变量检查（如“这个列表必须排序后才能二分查找”）。

·初学者误区：误以为 raise 异常类与 raise 异常实例有区别——其实对类对象 Python 会自动实例化它（raise Combust $\equiv$ raise Combust()）。还有人混淆 assert 与条件 raise：assert 是“开发期自检”（可能被 -O 关闭），对外部输入的验证应该用 if + raise。

代码分析

```
try:
    raise IndexError                # raise IndexError(...)
except IndexError:
    print('got exception')         #

assert 1 < 0, 'Not in this universe!' # → AssertionError
```

解释器如何处理：raise IndexError 的字节码 RAISEVARARGS 1 发现栈上是异常类而非实例，就调用 IndexError() 创建实例，然后进入与自动异常完全相同的分发流程：在当前帧异常表中查找匹配处理器，找到 except IndexError 即跳转并执行 print。assert 1 < 0 编译后是：先计算 1 < 0（得 False），条件为假则 LOADASSERTIONERROR + 压入消息字符串 + RAISEVARARGS 2——最终 traceback 显示 AssertionError: Not in this universe!，因为顶层没有处理器。

设计思想：assert 的英文原意是“声明/断言”——它不是在捕获错误，而是在声明你认为不可能发生的事情。把它写进代码等于给未来的读者（包括你自己）留下路标：这里的状态必须如此。

用户自定义异常 (User-Defined Exceptions)

英文原文

The raise statement demos in the prior section raised `IndexError`, a built-in exception defined in Python's built-in scope. As you'll learn later in this part of the book, you can also define new exceptions of your own that are specific to your programs. User-defined exceptions are coded with classes, which inherit from a built-in exception class—usually, the class named `Exception`:

```
python >>> class Combust(Exception): pass # User-defined exception
>>> def makePizza(): ... raise Combust() # Raise an instance
>>> try: ... makePizza() ... except Combust: # Catch class name ... print('got exception') ... got exception
```

>>> As you'll see in upcoming chapters, exception classes allow scripts to build exception categories, which can inherit behavior and have attached state information and methods, and an `as` clause on an `except` can gain access to the exception object itself. Exception classes can also customize their message text display.

ayed if they're not caught:

```
python >>> class Combust(Exception): ... def str(self):  
... return'Call the fire department!...'>>> raise Combust()  
Traceback (most recent call last): File "<stdin>", line 1, in <module> Combust: Call the fire department!  
!...  
  
>>>
```

中文翻译

前一节 raise 语句的演示引发的是 IndexError——一个定义在 Python 内置作用域中的内置异常。正如你将在本部分后面学到的，你也可以定义属于自己的、专属于你的程序的新异常。用户自定义异常用类（class）编写，这些类继承自一个内置异常类——通常继承名为 Exception 的类：python >>> class Combust(Exception): pass # 用户自定义异常>>> def makePizza(): ... raise Combust() # 引发一个实例>>> try: ... makePizza() ... except Combust: # 用类名捕获... print('got exception') ... got exception >>> 正如你将在接下来的章节中看到的，异常类允许脚本构建异常类别（category），这些类别可以继承行为、携带附加的状态信息和方法，而 except 上的 as 子句可以访问异常对象本身。异常类还可以定制它们未被捕获时显示的消息文本：python >>> class Combust(Exception): ... def str(self): ... return'Call the fire department!...'>>> raise Combust()
Traceback (most recent call last): File "<stdin>", line 1, in <module> Combust: Call the fire department!
!... >>>

深度理解

- 核心概念：自定义异常 = 定义一个继承 Exception 的类。异常与类的继承体系绑定，这使得 `except Combust` 不仅捕获 Combust，还能捕获 Combust 的所有子类——异常分类学由此而来。
- 底层实现：`raise Combust()` 创建异常对象后，分发机制用 `type(exception)` 与 `except` 子句的类做 `issubclass` 匹配。内置异常继承树根是 `BaseException` (`SystemExit`、`KeyboardInterrupt` 直接继承它)，日常业务异常应继承 `Exception`。`traceback` 末尾 `Combust: Call the fire department!...` 的 "类型: 消息" 格式，消息部分来自对异常实例调用 `str()`——所以重写 `str` 就能定制显示文本。
- 设计原因：为什么非要类？因为光有名字不够——异常需要携带数据（状态信息、方法），还需要支持继承分组（比如 `except DataError` 一次捕获所有数据类错误）。这正是"需要先学类才能学自定义异常"的原因。
- 实际问题：工程上应针对错误的类别（而不是每个具体错误）定义异常：如 `class NetworkError(Exception)`，其下再细分 `TimeoutError`、`ConnectionError`。调用方捕获 `NetworkError` 就覆盖了整个网络故障族。
- 初学者误区：自定义异常时直接继承 `Exception` 是正确姿势；直接继承 `BaseException` 是错误姿势（会连 `KeyboardInterrupt` 一并拦截）。另外注意 `except Combust` 写的是类名而非实例——用 `except Combust:` 与 `except Combust():` 都能工作，但前者是惯例。

代码分析

```
class Combust(Exception): pass #

def makePizza():
    raise Combust() #

try:
    makePizza()
except Combust: #
    print('got exception')

class Combust(Exception):
    def __str__(self): #
        return 'Call the fire department!...'
    raise Combust()
# Combust: Call the fire department!...
```

解释器如何处理：class Combust(Exception) 定义了一个继承自 Exception 的类（mro 链：Combust → Exception → BaseException → object）。raise Combust() 创建实例并抛出；makePizza() 帧无处理器被弹出；模块层异常表发现 except Combust 与异常类型匹配（issubclass(Combust, Combust) 为真），跳转执行 print。第二个例子中 raise Combust() 未被捕获，顶层默认处理器打印 traceback 时对异常调用 str()，得到自定义文本 Call the fire department!...，所以最后一行是 Combust: Call the fire department!...。

设计思想：class Combust(Exception): pass 用三个字符（: pass）就定义了一种全新的异常类型——这是 Python "约定优于配置"的极致体现。类定义本身同时是"类别"，

捕获时的 `issubclass` 匹配让异常体系天然支持多级分类。

终止动作 (Termination Actions)

英文原文

Finally, try statements can say "finally"—that is, they may include finally blocks. These look like except handlers for exceptions, but the try/finally combination specifies termination actions that always execute "on the way out," regardless of whether an exception occurs in the try block or not. Continuing our REPL session:

```
python >> try:
...     fetcher(food, 4) ... finally: # Termination actions ...
print('after fetch') ... 'a'after fetch >>
```

Here, if the try block finishes without an exception, the finally block will run, and the program will resume after the entire try. In this case, this statement seems a bit silly—we might as well have simply typed the `print` right after a call to the function, and skipped the try altogether:

```
python fetcher(food, 4)
print('after fetch')
```

There is a problem with coding this way, though: if the function call raises an exception, the `print` will never be reached. The try/finally combination avoids this pitfall—when an exception does occur

ur in a try block, finally blocks are executed while the program is being unwound:

```
python >> def after():
```

```
... try: ... fetcher(food, 5) ... finally: # Always run ... print('after fetch') ... print('after try?') # Run only if no exception
>> after() after fetch Traceback (most recent call last): File "<stdin>", line 1, in <module> File "<stdin>", line 3, in after File "<stdin>", line 2, in fetcher IndexError: string index out of range >>
```

Here, we don't get the "after try?" message because control does not resume after the try statement when an exception occurs. Instead, Python jumps back to run the finally action and then propagates the exception up to a prior handler (in this case, to the default handler at the top). If we change the call inside this function so as not to trigger an exception, the finally code still runs, but the program continues after the try:

```
python >> def after():
```

```
... try: ... fetcher(food, 4) ... finally: # Both run if no exception
... print('after fetch') ... print('after try?') >> after() after fetch after try?
>>
```

In practice, except clauses in a try are useful for catching and recovering from exceptions, and finally clauses come in handy to guarantee that termination acti

ons will fire regardless of any exceptions that may occur in the try block's code.

For instance, you might use try/except combinations to catch errors raised by code that you import from a third-party library, and try/finally combos to ensure that calls to close files or terminate server connections are always run. We'll code some such practical examples later in this part of the book.

Although they serve conceptually distinct purposes, you can also mix except and finally clauses in the same try statement—the finally is run on the way out regardless of whether an exception was raised, and regardless of whether the exception was caught by an except clause. Such combos have rules you'll meet in the next chapter.

As you'll also learn in the next chapter, Python provides an alternative to the try/finally mix when using some types of built-in and user-defined objects. The with statement runs a context manager object's methods to guarantee that termination actions occur, irrespective of any exceptions in its nested block:

```
python >> with open('pizzarobot.txt','w') as file: # Always close file on exit
```

```
... file.write('Catch fire!\n')
```

Although this option requires fewer lines of code, it's applicable only when processing certain object types, so try/finally is a more general termination structure, and is often simpler than

coding a class in cases where with is not already supported. On the other hand, with may also run startup actions too, and supports user-defined context management code with access to Python's full OOP toolset. To see how, let's move on to the next chapter.

中文翻译

最后, try 语句可以说"finally"——也就是说, 它们可以包含 finally 块。这些块看起来像捕获异常的 except 处理器, 但 try/finally 组合指定的是终止动作 (termination action): 无论 try 块中是否发生异常, 它们都会"在出去的途中" (on the way out) 始终执行。继续我们的 REPL 会话:

```
python >>> try: ... fetcher(food, 4) ... finally: # 终止动作... print('after fetch') ... 'a'after fetch >>>
```

在这里, 如果 try 块没有发生异常地正常结束, finally 块就会运行, 程序会在整个 try 之后恢复执行。这种情况下, 这条语句看起来有点蠢——我们不如直接在函数调用后面打 print, 干脆跳过 try 算了:

```
python fetcher(food, 4) print('after fetch')
```

不过, 这样编码有个问题: 如果函数调用引发了异常, print 就永远不会执行到。try/finally 组合避免了这个陷阱——当异常确实发生在 try 块中时, finally 块会在程序被回卷 (unwound) 的过程中执行:

```
python >>> def after(): ... try: ... fetcher(food, 5) ... finally: # 始终运行... print('after fetch') ... print('after try?') # 仅在没有异常时运行>>> after() after fetch Traceback (most recent call last): File "<stdin>", line 1, in <module> File "<stdin>", line 3, in after File "<stdin>", line 2, in fetcher IndexError: string index out of range >>>
```

在这里, 我

们没有看到 "after try?" 消息，因为发生异常时控制流并不会在 try 语句之后恢复。相反，Python 会跳回去运行 finally 动作，然后把异常传播（propagates）到更早的处理器（这里是顶层的默认处理器）。如果我们把这个函数内部的调用改成不会触发异常的形式，finally 代码依然会运行，但程序会在 try 之后继续：

```
python >>> def after(): ... try : ... fetcher(food, 4) ... finally: # 没有异常时两者都运行... print('after fetch') ... print('after try?') >>> after() after fetch after try? >>>
```

实践中，try 中的 except 子句用于捕获异常并从中恢复，而 finally 子句则用来保证：无论 try 块代码中可能发生什么异常，终止动作都会触发。例如，你可以用 try/except 组合捕获从第三方库导入的代码引发的错误，用 try/finally 组合确保关闭文件或终止服务器连接的调用总是执行。本部分后面我们会编写一些这样的实用示例。虽然它们在概念上用途不同，但你也可以在同一个 try 语句中混合使用 except 和 finally 子句——无论是否引发了异常，也无论异常是否被 except 子句捕获，finally 都会在"出去的路上"运行。这类组合有一些规则，你将在下一章遇到。正如你也将在下一章学到的，在使用某些类型的内置和用户自定义对象时，Python 提供了 try/finally 混合方案的替代品。with 语句会运行一个上下文管理器（context manager）对象的方法，以保证无论其嵌套块中是否有异常，终止动作都会发生：

```
python >>> with open('pizzarobot.txt','w') as file: # 退出时总是关闭文件... file.write('Catch fire!\n') 虽然这个方案需要的代码行数更少，但它只适用于处理某些类型的对象，所以 try/finally 是更通用的终止结构；在 with 尚不支持的情况下，它往往也比编写一个类更简单。另一方面，with 还可以运行启动动作（start
```

up action) , 并且支持用户自定义的上下文管理代码, 可以访问 Python 完整的 OOP 工具集。想看看怎么做? 让我们进入下一章。

深度理解

·核心概念: finally 是保证条款——"无论发生什么, 这一段必须执行"。它不捕获异常, 异常会在 finally 执行后继续传播。这就是它与 except 的本质区别: except 是"拦截+处理", finally 是"路过必办手续"。

·底层实现: 异常回卷时, 解释器并非直接把异常抛到上层, 而是沿途执行每个栈帧上活动的 finally 块。字节码层面, 3.11 之前的实现是 try-finally 块被编译成 SETUPFINALLY 与跳转表, 异常时执行 finally 代码 (一条 ENDFINALLY 会重新抛出保存的异常); 3.11+ 用异常表 (exception table) 记录每个 finally 的范围, 异常路径上先执行 finally 再继续传播。所以 after() 中先打印 after fetch, 随后异常继续上浮到顶层打印 traceback——顺序铁打不变。

·设计原因: 资源释放 (关闭文件、断开连接、释放锁) 必须在所有路径上执行——包括异常路径、return 路径、break/continue 路径。把它们写在"异常后的语句"位置会漏执行; finally 用一条语法解决所有路径的覆盖问题。

·实际问题: 作者给出黄金分工——try/except 用于捕获第三方库错误 (业务恢复), try/finally 用于保证收尾 (资源清理); with 是更简洁的收尾工具但只适用于实现了上下文管理协议的对象。工程惯例: 新版代码能 with 就

with，需要细粒度控制再用 try/finally。

·初学者误区：误以为 finally 会"吞掉"异常——不会，异常在 finally 之后继续传播（上面的 traceback 证明了）。还有人不明白 finally 里 return 会把异常"顶掉"——finally 中若执行 return，会中断异常传播（这是坑，应避免）。另外注意：try/finally 没有 except 也能独立存在，它俩不是绑定关系。

代码分析

```
def after():
    try:
        fetcher(food, 5)          # IndexError
    finally:                       #
        print('after fetch')      #
    print('after try?')           #

after()
#
# after fetch
# Traceback ... IndexError: string index out of range

with open('pizzarobot.txt', 'w') as file:
    file.write('Catch fire!\n')
```

解释器如何处理：after() 中 fetcher(food, 5) 抛出 IndexError。异常表显示该行位于 finally 保护区段内，于是解释器先把异常暂存，跳转执行 print('after fetch')（此时原始异常"挂起"在解释器内部状态中），finally 块结束后重新抛出暂存的异常，继续沿栈传播到顶层默认处理器。所以输出顺序必然是"after fetch"在前、traceback 在后。若改为 fetcher(food, 4)，一切正常时 finally 照常执行

，指令指针继续走到 `print('after try?')`，两条消息都输出。 `with` 语句在字节码层面等价于 `try/finally` 加 `enter/exit` 方法调用：进入时调用 `enter`（打开文件），离开（无论是否异常）调用 `exit`（关闭文件）。

设计思想：`try/finally` 把"清理"从业务代码中剥离出来——你写"做什么"，`finally` 声明"无论如何都要收尾"，两者职责分离。而 `with` 更进一步，把"进入/退出"协议封装进对象（上下文管理器），调用方连 `finally` 都不用写，这为第 34 章的上下文管理器与 OOP 的融合埋下伏笔。

本章小结 (Chapter Summary)

英文原文

And that is the majority of the exception story; exceptions really are a simple tool. To summarize, Python exceptions are a control-flow device. They may be raised by Python, or by your own programs. In both cases, they may be ignored (to trigger the default error handler), or caught by `try` statements (to be processed by your code). The `try` statement comes in logically distinct forms that can be combined—one that handles exceptions, and one that runs finalization code regardless of whether exceptions occur or not. Python's `raise` and `assert` statements trigger exceptions on demand—both built-ins and new exceptions we define with classes—and the `with` statement is an alternative w

ay to ensure that termination actions are carried out for objects that support it. In the rest of this part of the book, we'll fill in some of the details about the statements involved, examine the other sorts of clauses that can appear under a try (spoiler: it also allows an else for the no-exception case), and discuss class-based exception objects. The next chapter begins our tour by taking a closer look at the statements we introduced here. Before you turn the page, though, here are a few quiz questions to review.

中文翻译

以上就是异常故事的大部分内容了；异常真的只是一个简单的工具。总结一下：Python 异常是一种控制流装置。它们可以由 Python 引发，也可以由你自己的程序引发。无论哪种情况，它们都可以被忽略（从而触发默认错误处理器），也可以被 try 语句捕获（由你的代码处理）。try 语句具有逻辑上不同、可以组合的几种形式——一种处理异常，一种在无论是否发生异常的情况下都运行终结（finalization）代码。Python 的 raise 和 assert 语句可以按需触发异常——包括内置异常和我们用类定义的新异常——而 with 语句是另一种确保对支持它的对象执行终止动作的方式。在本部分的其余章节中，我们将补充一些相关语句的细节，考察 try 下面还可以出现的其他种类子句（剧透：它还允许一个处理“无异常情况”的 else），并讨论基于类的异常对象。下一章将更仔细地研究我们这里介绍的语句，开始我们的旅程。不过，在你翻页之前，这里有几道测验题供你复习。

深度理解

·核心概念：本章总结把全部内容浓缩为三句话——异常是控制流装置；要么忽略（默认处理器）、要么捕获（try）；try 有"处理异常"和"保证收尾"两种可组合的形式，raise/assert 按需触发，with 是收尾的语法糖替代品。

·底层实现："可以组合"暗示 try/except/finally/else 在字节码层面对应同一套异常表机制，只是不同的子句组合产生不同的跳转与清理行为——这也是下一章"组合规则"的由来。

·设计原因：作者预告 try 还有 else 子句（无异常时执行），把悬念留到第 34 章，保持本章"基础"的轻量定位——先建立心智模型，再填充细节。

·实际问题：学完本章你已经能看懂程序崩溃的 traceback、能用 try/except 让脚本存活、能用 try/finally 与 with 保证资源释放——这三项足以覆盖绝大多数日常异常处理需求。

·初学者误区：以为"异常部分"就这些——其实这只是地图；try/else、try 组合规则、异常类层级、raise from 链式异常、自定义上下文管理器都还在第 34 章等着。

Test Your Knowledge: Quiz (测验)

英文原文

1. Name three things that exception processing is good for.
2. What happens to an exception if you don't

do anything special to handle it? 3. How can your script recover from an exception? 4. Name two ways to trigger exceptions in your script. 5. Name two ways to specify actions to be run at termination time, whether an exception occurs or not.

中文翻译

1. 说出异常处理的三项用途。2. 如果你不采取任何特殊措施处理异常，会发生什么？3. 你的脚本如何从异常中恢复？4. 说出两种在你的脚本中触发异常的方式。5. 说出两种指定“终止时刻要执行的动作”（无论是否发生异常）的方式。

Test Your Knowledge: Answers (答案)

英文原文

1. Exception processing is useful for error handling, termination actions, and event notification. It can also simplify the handling of special cases and can be used to implement alternative control flows as a kind of structured "go-to" operation. In general, exception processing also cuts down on the amount of error-checking code your program may require—because all errors filter up to handlers, you may not need to test the outcome of every operation (see this chapter's sidebar "Why You Will Care: Error Checks" for an illustration). 2. Any uncaught exception eventually filters up t

o the default exception handler Python provides at the top of your program. This handler prints the familiar error message and shuts down your program. 3. If you don't want the default message and shutdown, you can code try statements with except clauses to catch and recover from exceptions that are raised within its nested code block. Once an exception is caught, the exception is terminated and your program continues after the try. 4. The raise and assert statements can be used to trigger an exception, exactly as if it had been raised by Python itself. In principle, you can also raise an exception by making a programming mistake, but that's not usually an explicit goal! 5. The try statement with a finally clause can be used to ensure actions are run after a block of code exits, regardless of whether the block raises an exception or not. The with statement can also be used to ensure termination actions are run, but only when processing object types that support it.

中文翻译

1. 异常处理适用于错误处理、终止动作和事件通知。它还可以简化特殊情况的处理，并可以作为一种结构化的 "goto" 操作来实现替代控制流。总的来说，异常处理还能减少程序可能需要的错误检查代码量——因为所有错误都会"上浮"到处理器，你可能不需要测试每一个操作的结果（参见本章的专栏"Why You Will Care: Error Checks"中的示例）。
2. 任何未捕获的异常最终都会上浮到 Python 在程序顶层

提供的默认异常处理器。这个处理器会打印熟悉的错误消息并关闭你的程序。3. 如果你不想要默认的消息和关闭行为，可以编写带 `except` 子句的 `try` 语句，捕获并从中嵌套代码块中引发的异常中恢复。一旦异常被捕获，异常即告终结，你的程序在 `try` 之后继续执行。4. `raise` 和 `assert` 语句可以用来触发异常，就仿佛异常是 Python 自己引发的一样。原则上，你也可以通过制造一个编程错误来引发异常，但那通常不是明确的目标！5. 带 `finally` 子句的 `try` 语句可以用来确保代码块退出后动作一定会运行，无论该块是否引发异常。`with` 语句也可以用来确保终止动作的运行，但只适用于处理支持它的对象类型。

深度理解

·核心概念：五道测验正好覆盖本章全部知识点：异常用途（答案 1）、默认处理器行为（答案 2）、捕获恢复（答案 3）、触发方式（答案 4）、收尾保证（答案 5）——这是作者给读者的“考点清单”，能脱口而出这五点说明本章达标。

·底层实现：答案 2 中“shuts down your program”在底层是默认处理器（`sys.excepthook`）打印 `traceback` 后，`sys.exit()` 或等效机制终止进程；答案 4 强调“就仿佛 Python 自己引发的一样”——因为 `raise/assert` 与自动异常共享完全相同的分发管道。

·设计原因：作者故意把“编程错误也是一种 `raise` 途径”（答案 4 的彩蛋）写成幽默提醒，既教了知识也暗示了调试策略：复现 bug 时可以临时在可疑点加 `raise` 验证路径。

·实际问题：这五问可作为面试自测题——能答全，说明异常基础已建立，可以放心进入第 34 章的 try 语句细节、else 子句与异常对象。

·初学者误区：答案 3 的关键短语是“一旦异常被捕获，异常即告终结”——捕获后异常生命周期结束，程序从 try 之后继续，而不是从异常点继续。别漏掉答案 5 的限定：“仅适用于支持它的对象类型”——with 不是万能的，try/finally 更通用。

专栏：为什么你在意——错误检查 (Why You Will Care: Error Checks)

英文原文

One way to see how exceptions are useful is to compare coding styles in Python and languages without exceptions. For instance, if you want to write robust programs in the C language, you generally have to test return values or status codes after every operation that could possibly go astray, and propagate the results of the tests as your programs run:

```
c doStuff() { / C program /  
  
    if (doFirstThing() == ERROR) / Detect errors everywhere /  
    return ERROR; / even if not handled here /  
    if (doNextThing() == ERROR) return ERROR; ... return doLastThing(); }  
main() {
```

```
if (doStuff() == ERROR) badEnding(); else goodEnding();}
```

In fact, realistic C programs often have as much code devoted to error detection as to doing actual work. But in Python, you don't have to be so methodical (and neurotic!). You can instead wrap arbitrarily vast pieces of a program in exception handlers and simply write the parts that do the actual work, assuming all is normally well:

```
python def doStuff(): # Python code doFirstThing() #  
We don't care about exceptions here, doNextThing()  
# so we don't need to detect them
```

```
... doLastThing()
```

```
if name == 'main': try: doStuff() # This is where we care  
about results,
```

```
except: # so it's the only place we must check badEnding()
```

```
else: # See the next chapter for else goodEnding()
```

Because control jumps immediately to a handler when an exception occurs, there's no need to instrument all your code to guard for errors, and there's no extra performance overhead to run all the tests. Moreover, because Python detects errors automatically, your code often doesn't need to check for errors in the first place. The upshot is that exceptions let you largely ignore the unusual cases and avoid error-checking code

e that can distract from your program's purpose.

中文翻译

要看清异常的用处，一种方法是比较 Python 与没有异常的语言的编码风格。例如，如果你想用 C 语言编写健壮的程序，通常必须在每个可能出问题的操作之后测试返回值或状态码，并在程序运行过程中传播测试的结果：`c doStuff() { / C 程序 / if (doFirstThing() == ERROR) / 处处检测错误 / return ERROR; / 即使不在这里处理 / if (doNextThing() == ERROR) return ERROR; ... return doLastThing(); }`
`main() { if (doStuff() == ERROR) badEnding(); else goodEnding(); }`事实上，真实的 C 程序常常把与“实际干活”一样多的代码投入到错误检测上。但在 Python 中，你不必如此一丝不苟（也不必如此神经质！）。相反，你可以把程序中任意大的部分包进异常处理器，只编写那些实际干活的部分，并假设一切正常：`python def doStuff(): # Python 代码doFirstThing() # 我们在这里不关心异常，doNextThing() # 所以无需检测它们... doLastThing() if name == 'main': try: doStuff() # 这才是我们关心结果的地方，except: # 所以也是唯一必须检查的地方badEnding() else: # else 详见下一章goodEnding()`因为异常发生时控制流会立即跳转到处理器，所以不需要在你的所有代码中布防错误检查，也不需要为了运行所有这些测试而付出额外性能开销。此外，由于 Python 会自动检测错误，你的代码往往一开始就不需要检查错误。结论是：异常让你可以很大程度上忽略不寻常的情况，避免那些会分散你对程序目标注意力的错误检查代码。

深度理解

·核心概念：C 风格的"处处检测" (everywhere check) v s Python 风格的"集中恢复" (catch only where you can recover) ——错误检测代码与业务代码的比例从"1:1" 降到"只在边界处检查一次"。

·底层实现：C 语言错误检测是运行时成本（每次调用后都有一次比较+分支），且遗漏一处检查就静默出错；Python 异常检测成本只在错误发生时支付——正常路径没有额外的比较指令，这是专栏所说"没有额外性能开销"的底层原因。

·设计原因：为什么 C 没有异常？历史原因——C 诞生于 1970 年代初的 UNIX 时代，语言本体极简；异常靠程序员纪律（返回码）约定。Python 把异常内置进语言（1989 年起设计，吸收自 C++/Modula 等），是因为作者深知"程序员纪律"不可靠。

·实际问题：if name == 'main': 是脚本入口惯例；except: (裸 except) 捕获一切，else 子句在无异常时运行 goodEnding()。这个模式就是真实项目的"主函数骨架"：入口处一个 try 包住整个流程，失败统一 badEnding。

·初学者误区：把 C 的思考方式带进 Python——对每个函数调用都 if 判断返回值；或反过来，把 Python 的裸 except: 用成"掩饰所有错误"。专栏暗示的正确姿势：正常路径不检查，边界路径才捕获；裸 except 通常要避免（第 34 章细讲）。

代码分析 (C 与 Python 对照)

```
doStuff()
{
    /* C */
    if (doFirstThing() == ERROR)    /* */
        return ERROR;             /* */
    if (doNextThing() == ERROR)
        return ERROR;
    ...
    return doLastThing();
}

main()
{
    if (doStuff() == ERROR)
        badEnding();
    else
        goodEnding();
}
```

```
def doStuff():                    # Python
    doFirstThing()               #
    doNextThing()                #
    ...
    doLastThing()

if __name__ == '__main__':
    try:
        doStuff()               #
    except:                      #
        badEnding()
    else:                        # else
        goodEnding()
```

解释器如何处理：C 版本中 doStuff 每一步都必须显式检查返回值并逐层向上传递——任一环节忘记检查，错误就静默扩散。Python 版本中 doStuff() 内部没有一条错误

检查指令；任何一步出错，异常自动沿调用栈上浮，try 块的异常表直接捕获，跳转 badEnding()；全部成功则执行 else 块的 goodEnding()。性能上，C 的正常路径要为每次调用付出比较分支成本，Python 正常路径零额外成本——异常只在“非常路径”上才发生跳转。

设计思想：这个对照是全书最有力的“为什么用 Python”论证之一：异常把错误检测的负担从每个调用点集中到恢复点，让代码读起来像“业务故事”而不是“防御工事”。作者用 # This is where we care about results 注释点明哲学：只在你关心结果的地方检查。

脚注 (Footnote 1)

英文原文

¹ For any computer scientists in the audience, true backtracking is not part of the Python language. Backtracking undoes computations before it jumps back, but Python exceptions do not: local variables in open function calls run by the try are simply discarded, but changes made to globals and objects are retained (see the demo ahead). Even the generator functions and expressions we met in Chapter 20 don't do full backtracking—they simply respond to next(G) requests by restoring saved state and resuming. For more on backtracking, try books on AI or the Prolog or Icon programming languages.

中文翻译

¹ 对于读者中的计算机科学家们：真正的回溯（backtracking）不是 Python 语言的一部分。回溯在跳回之前会撤销计算，但 Python 异常不会：try 运行的、尚未结束的函数调用中的局部变量被直接丢弃，但对全局变量和对象的修改会被保留（参见前面的演示）。即使是我们在第 20 章遇到的生成器函数和表达式也不做完整的回溯——它们只是响应 next(G) 请求，恢复保存的状态并继续执行。想了解更多回溯，可以看看关于 AI 的书籍或 Prolog、Icon 编程语言。

深度理解

·核心概念：Python 异常与“逻辑编程中的回溯”是两回事：
回溯 = 撤销副作用再跳转；Python 异常 = 只拆除调用栈、保留所有已发生的副作用。生成器用 yield 保存/恢复状态，也不是回溯。

·底层实现：回溯需要“逆向执行”（undo）——修改过的变量、对象都要恢复原值；Python 解释器没有维护任何“变更日志”，无法撤销堆上对象的变化，所以只能撤销“栈上局部变量”（因为它们随帧一起销毁）。

·设计原因：完整回溯的实现成本极高（需要存储每一层的完整状态快照），Python 选择了“跳转 + 保留副作用”的务实方案——足够应付错误恢复，又不需要虚拟机支持事务性回滚。

·实际问题：理解这一点能解释很多怪异现象：try 块里改了全局变量，异常后依然生效（前面 modder 的例子）；

这正是"异常不是事务"的底层原因——需要回滚时（如写文件一半出错要删掉半成品），必须自己在 except 里补回滚逻辑。

·初学者误区：把异常当"撤销键"；以为出了异常"程序恢复原状"。脚注明确指出：只有局部变量被丢弃，其余一切照旧——写回滚代码是你的责任，不是解释器的责任。

本章总结

技术拓展 (Technical Expansion)

·实际项目中的应用场景：异常是"分层架构"的粘合剂——底层函数只管抛（raise ValueError），中间层不处理（让它上浮），顶层统一记录日志并按错误类别分流（except 具体类 → 降级重试 → 兜底）。Web 框架（Flask/Django）把异常映射成 HTTP 错误码、ORM 把数据库错误包装成业务异常，靠的都是本章的基础机制。真实服务器代码里 try/finally（或 with）保证连接关闭的写法，就是本章"终止动作"的直接应用。

·与其他语言的区别：

语言	异常机制	关键差异
C	无异常，靠返回值/errno	错误检测代码约占一半；遗漏检查即静默出错
C++	try/catch/throw，可抛出任意类型	可抛 int/char 等，异常对象无强制基类；RAII 用析构函数保证清理

Java	checked exception (必须声明)	强制签名 throws, 编译器逼你处理; Python 刻意没有 checked exception
Python	一切异常继承 BaseException; try/except/else/finally	异常是类层级, except 用子类匹配; 无强制声明; finally/with 保证清理

· Python 发展历史背景: Python 从 1.5 版 (1997 年) 起异常对象采用类层级设计, 此前是字符串异常。with 语句与上下文管理器协议是 PEP 343 (2005 年, Guido 亲自推动) 引入。except 与 ExceptionGroup 是 Python 3.11 (2022 年) 的新特性, 用于并发场景的多异常合并——作者建议初学者推迟学习, 正因为它解决的是边缘问题。作者写本章时基于 Python 3.12。

· 高级开发者需要掌握的相关知识: 异常链 (raise ... from ...) 与 cause/context 调试; sys.excepthook 自定义未捕获异常处理器; 警告 (warnings 模块) 与异常的取舍; traceback 模块的 printexc(); 上下文管理器协议 enter/exit (第 34 章); 在 -O 优化下 assert 被删除的实践含义; 以及在多线程/异步代码中异常传播边界 (每个线程/任务各自分发异常)。

学习建议 (Learning Advice)

· 重要程度: (5/5 星) ——异常处理是 Python 程序健壮性的基石, 是任何真实项目 (脚本、服务、库) 都离不开的核心技能, 也是本书后续章节 (第 34、35 章) 的直接基础。

· 应该掌握到什么程度: ① 熟练阅读和解释 traceback (

从下往上看：异常类型→消息→调用链）；② 能独立写出 try/except（精确捕获）、try/finally（保证收尾）、raise（快速失败）、assert（开发期自检）、with（资源管理）这五种形态；③ 理解“异常只拆除栈帧、不撤销堆上副作用”；④ 能区分异常传播与回溯。

·后续应该学习哪些内容：下一章（第 34 章）深入 try 语句的完整语法与组合规则（try/except/else/finally 的顺序、except Exception as e、裸 except 的坑、raise 与异常链）；再下一章学习基于类的异常对象设计（异常层级、状态信息、str 定制）；随后可学习上下文管理器协议（with 的自定义实现），最终在实践中结合日志模块（logging）构建完整的错误处理体系。